

Pandas

Revision Sheet — Quick Reference

What is Pandas?

Pandas is Python's table library — like Excel but automated and scriptable.

NumPy works with numbers only. Pandas works with rows + columns (mixed types).

It does what Excel does manually AND what SQL does with queries — inside Python.

Real DA work is 70% Pandas. It is the most important library you will learn.

Why it matters:

- Load any CSV/Excel file in one line: `pd.read_csv()`
- Filter, group, merge, reshape data without SQL or Excel
- Handles missing values, duplicates, type casting cleanly
- Foundation for every data analysis and ML pipeline in Python

```
import pandas as pd

df = pd.read_csv('data.csv') # load a dataset
df.head()                   # preview first 5 rows
df.info()                   # check dtypes + nulls
df.describe()               # statistical summary
```

01 Series & DataFrame

Series = one column (1D) with an index. DataFrame = full 2D table (multiple columns).

In real work you almost always use DataFrames. Series appear when selecting one column.

```
s = pd.Series([10, 20, 30, 40])      # 1D - index + values

df = pd.DataFrame({
    'name':  ['Alice', 'Bob', 'Charlie'],
    'age':   [25, 30, 35]
})                                     # 2D - rows + columns
```

02 Reading Data

pd.read_csv() loads any CSV into a DataFrame. Use nrows/usecols to load partial data.

Always save with df.to_csv('file.csv', index=False) to avoid a junk index column.

```
df = pd.read_csv('students.csv')      # full file
df = pd.read_csv('students.csv', nrows=3)  # first 3 rows only
df = pd.read_csv('students.csv', usecols=['name', 'marks']) # specific cols

df.to_csv('output.csv', index=False)    # save back to CSV
```

03 Viewing Data

First thing after loading any dataset. These 4 lines are your standard starting point.
info() shows dtypes + null counts. describe() gives stats on numeric columns only.

```
df.head()          # first 5 rows | df.head(3) # first 3
df.tail(2)         # last 2 rows
df.info()          # column names, dtypes, non-null counts
df.describe()      # count, mean, std, min, 25%, 50%, 75%, max
df.shape           # (rows, cols) – e.g. (100, 5)
```

04 Selecting Data – loc & iloc

Select columns with df['col'] (Series) or df[['a','b']] (DataFrame).

loc uses labels. iloc uses integer positions. Rule: loc=names, iloc=numbers.

```
df['name']          # single col → Series
df[['name', 'marks']] # multi col → DataFrame

df.loc[0]           # row 0 by label
df.loc[0, 'name']   # row 0, name col
df.loc[0:2, 'name':'marks'] # rows 0-2, cols name to marks

df.iloc[0, 1]       # row 0, col 1 by position
df.iloc[0:2, 0:2]   # slice by position
```

05 Filtering Rows

Same as NumPy boolean masking but on a DataFrame. Use & for AND, | for OR.
isin() filters multiple values cleanly without chaining multiple OR conditions.

```
df[df['marks'] > 80]                # single condition
df[df['city'] == 'Delhi']          # exact match
df[(df['city'] == 'Delhi') & (df['marks'] > 80)] # AND
df[(df['city'] == 'Delhi') | (df['city'] == 'Mumbai')] # OR
df[df['city'].isin(['Delhi', 'Pune'])]      # isin()
```

06 Missing Values

Real datasets always have NaN. Find them with isnull(). Drop or fill based on % missing.
fillna() returns a NEW object — always assign back: df['col'] = df['col'].fillna(x)

```
df.isnull().sum()                  # count NaN per column
df.isnull().sum().sum()            # total NaN in DataFrame

df.dropna()                       # drop rows with any NaN
df.dropna(subset=['marks'])        # drop only if marks is NaN

df['marks'].fillna(df['marks'].mean()) # fill with mean
df['city'].fillna('Unknown')         # fill strings
df['marks'].ffill()                  # use previous row value
df['marks'].bfill()                  # use next row value

# dropna → missing < 5% | fillna → can't lose data
```

07 GroupBy

Split data into groups, apply a function, get results. Like SQL GROUP BY.
Use `.agg()` for multiple aggregations at once — essential for DA interviews.

```
df.groupby('city')['sales'].sum()           # sum by city
df.groupby('city')['sales'].agg(['sum', 'mean', 'count'])
df.groupby('city').agg(
    total=('sales', 'sum'), avg=('sales', 'mean'))
```

08 Merge

Combine two DataFrames on a common column. `how=` controls which rows to keep.

```
pd.merge(df_orders, df_customers, on='customer_id', how='inner')
# inner → only matching rows (104 & 105 dropped)
# left  → all of left + NaN where no match on right
# right → all of right + NaN where no match on left
# outer → everything from both sides, NaN where missing
```

09 Pivot Table

Like `groupby` but output is a 2D matrix — rows vs columns, values inside.
Use when comparing two dimensions (e.g. region × product). `groupby` gives flat list.

```
df.pivot_table(
    values='sales', # column to aggregate
    index='region', # becomes rows
    columns='product', # becomes columns
    aggfunc='sum', # how to aggregate
    fill_value=0 # replace NaN with 0
)
# product    A    B
# region
# North     220  150
# South     200  300
```

Topics Covered

Series & DataFrame • Read CSV • View • loc/iloc • Filter • Missing • GroupBy • Merge • Pivot